

NEXUS-CORP

Arquitectura de Inteligencia Organizacional

ESPECIFICACIÓN TÉCNICA – FASE 2

Knowledge-Based Decision Support System (KBDSS)

Documento:	ETS-KBDSS-F2-2025
Versión:	2.0.0
Clasificación:	Uso Interno / Restringido
Estado:	Técnico
Fecha de Emisión:	2026
Próxima Revisión:	2026
Responsable Técnico:	Arquitectura de Sistemas – Nexus-Corp
Revisado por:	Comité de Gestión del Conocimiento
Aprobado por:	Dirección de TI – Nexus-Corp

ÍNDICE GENERAL

ÍNDICE GENERAL.....	2
1. Introducción	5
1.1 Contexto Organizacional	5
1.2 Alcance del Documento.....	5
2. Propósito del Sistema	6
2.1 Objetivos Específicos	6
3. Alcance Técnico.....	7
3.1 Componentes del Sistema.....	7
3.2 Límites del Sistema	7
3.3 Exclusiones	7
4. Definiciones, Acrónimos y Abreviaturas	8
5. Referencias.....	10
6. Descripción General del Producto.....	11
6.1 Perspectiva del Producto.....	11
6.2 Funciones Principales.....	11
6.3 Características de los Usuarios	12
6.4 Restricciones del Sistema	12
6.5 Suposiciones y Dependencias.....	12
7. Especificación IEEE 830 – Requerimientos Funcionales.....	14
RF-01: Gestión de Usuarios	14
RF-02: Gestión de Roles	14
RF-03: Gestión de Reglas de Conocimiento.....	15
RF-04: Captura de Conocimiento	15
RF-05: Validación de Conocimiento	16
RF-06: Gestión de Escenarios.....	17
RF-07: Análisis What-If.....	17
RF-08: Generación de Recomendaciones.....	18
RF-09: Gestión de KPIs.....	18
RF-10: Retroalimentación Organizacional	19
RF-11: Historial de Decisiones	20
RF-12: Auditoría de Cambios	20
RF-13: Reportes.....	21
RF-14: Dashboard Ejecutivo.....	21
RF-15: Administración General	22
8. Requerimientos No Funcionales	24

9. Reglas de Negocio.....	26
9.1 Reglas de Inventario.....	26
9.2 Reglas de Demanda.....	26
9.3 Reglas de Riesgo.....	26
9.4 Reglas de Recomendaciones.....	26
9.5 Reglas de Retroalimentación.....	26
9.6 Reglas de Gestión del Conocimiento.....	27
10. Modelado UML.....	28
10.1 Diagrama de Casos de Uso.....	28
10.1.1 Código Mermaid – Diagrama de Casos de Uso.....	28
10.2 Especificación Textual de Casos de Uso.....	28
UC-01: Gestionar Usuarios.....	28
UC-03: Gestionar Reglas de Conocimiento.....	29
UC-07: Ejecutar Análisis What-If.....	29
UC-10: Registrar Retroalimentación.....	30
10.3 Diagrama de Clases.....	31
10.3.1 Código Mermaid – Diagrama de Clases.....	31
10.4 Diagramas de Secuencia.....	34
10.4.1 Secuencia: Captura de Conocimiento.....	34
10.4.2 Secuencia: Ejecución de Análisis What-If.....	34
10.4.3 Secuencia: Generación de Recomendación.....	35
10.4.4 Secuencia: Registro de Retroalimentación.....	35
10.5 Diagramas de Actividades.....	37
10.5.1 Actividad: Flujo de Validación de Reglas.....	37
10.5.2 Actividad: Proceso de Toma de Decisión Asistida.....	37
10.6 Diagrama de Componentes.....	38
10.7 Diagrama de Despliegue.....	38
11. Arquitectura del Sistema.....	40
11.1 Arquitectura General de Software.....	40
11.2 Clean Architecture – Capas del Sistema.....	40
11.2.1 Diagrama de Capas.....	40
11.2.2 Capa de Dominio (Domain Layer).....	40
11.2.3 Capa de Aplicación (Application Layer).....	41
11.2.4 Capa de Infraestructura (Infrastructure Layer).....	42
11.2.5 Capa de Presentación (Presentation Layer).....	42
11.3 Arquitectura del Motor de Reglas.....	42

- 12. Diseño de Base de Datos..... 44
 - 12.1 Modelo Conceptual..... 44
 - 12.2 Diagrama Entidad-Relación 44
 - 12.3 Diccionario de Datos – Tablas Principales..... 46
 - Tabla: usuarios 46
 - Tabla: reglas 46
- 13. Diseño del Backend 48
 - 13.1 Estructura de Módulos..... 48
 - 13.2 Flujo de Procesamiento de Decisiones 49
- 14. Arquitectura del Frontend 50
 - 14.1 Estructura de Componentes 50
 - 14.2 Navegación y Rutas..... 50
- 15. Especificación de APIs REST 52
 - 15.1 Convenciones Generales 52
 - 15.2 Endpoints Principales 52
 - 15.3 Estructura de Respuestas Estándar 53
 - Respuesta exitosa (200/201): 53
 - Respuesta de error (4xx/5xx): 54
 - 15.4 Códigos de Error del Sistema 54
- 16. Estrategia de Seguridad..... 56
 - 16.1 Autenticación – OAuth2 con JWT 56
 - 16.2 Autorización – Control de Acceso Basado en Roles (RBAC)..... 56
 - 16.3 Protección de Datos y Auditoría 57
- 17. Estrategia de Pruebas..... 58
 - 17.1 Niveles de Prueba 58
 - 17.2 Casos de Prueba Funcionales Seleccionados 58
 - 17.3 Matriz de Trazabilidad de Requisitos 59
- 18. Glosario Técnico 61
- 19. Conclusiones Técnicas 63
- 20. Recomendaciones Técnicas 63

1. Introducción

El presente documento constituye la Especificación Técnica Integral de la Fase 2 del proyecto Nexus-Corp: Arquitectura de Inteligencia Organizacional. Esta fase representa la definición exhaustiva de todos los componentes técnicos, funcionales, arquitectónicos y estructurales del Sistema de Gestión de Decisiones Basado en Conocimiento (Knowledge-Based Decision Support System – KBDSS) que será implementado en Nexus-Corp, empresa líder en distribución y logística tecnológica en la región centroamericana.

La Fase 1 estableció el marco conceptual, la visión del negocio y los objetivos estratégicos del sistema. La Fase 2, descrita en este documento, avanza hacia la especificación técnica detallada, cubriendo desde la arquitectura de software hasta el modelado de base de datos, pasando por los requerimientos funcionales y no funcionales, el modelado UML completo, la definición de las APIs REST, y las estrategias de seguridad, pruebas y despliegue.

El KBDSS de Nexus-Corp tiene como objetivo central transformar el conocimiento organizacional tácito en reglas de negocio explícitas, procesables y auditables, permitiendo a los tomadores de decisión disponer de una plataforma inteligente que apoye el análisis de escenarios, la generación de recomendaciones basadas en evidencia y el aprendizaje organizacional continuo.

1.1 Contexto Organizacional

Nexus-Corp opera en un entorno de alta complejidad logística, con cadenas de suministro multinivel, múltiples proveedores tecnológicos, variabilidad en la demanda y presión constante por eficiencia operativa. La ausencia de un sistema centralizado de gestión del conocimiento ha generado pérdida de conocimiento crítico ante rotación de personal, decisiones inconsistentes entre unidades de negocio y lentitud en la respuesta ante variaciones del mercado.

La implementación del KBDSS representa una inversión estratégica en capital intelectual organizacional, alineada con los objetivos de transformación digital definidos en el Plan Estratégico 2024–2028 de Nexus-Corp.

1.2 Alcance del Documento

Este documento cubre la totalidad de los artefactos técnicos requeridos para el diseño, desarrollo, prueba y despliegue del KBDSS, incluyendo: especificación IEEE 830, modelado UML, arquitectura Clean Architecture, diseño de base de datos relacional, diseño del backend en FastAPI (Python), diseño del frontend en React, especificación de APIs REST, estrategia de seguridad, plan de pruebas y glosario técnico.

2. Propósito del Sistema

El KBDSS tiene un propósito dual: operativo y estratégico. A nivel operativo, provee herramientas para la captura, validación, almacenamiento y recuperación del conocimiento organizacional en forma de reglas de negocio estructuradas. A nivel estratégico, actúa como un sistema de apoyo a la toma de decisiones, permitiendo a los directivos y analistas evaluar escenarios hipotéticos, comparar alternativas y generar recomendaciones fundamentadas en el conocimiento acumulado de la organización.

2.1 Objetivos Específicos

- Capturar el conocimiento tácito de expertos organizacionales y formalizarlo como reglas de negocio computables.
- Proveer un motor de inferencia que aplique reglas de conocimiento sobre datos en tiempo real para generar recomendaciones.
- Implementar un módulo de análisis What-If que permita simular escenarios bajo distintas hipótesis de negocio.
- Gestionar indicadores clave de desempeño (KPIs) y correlacionarlos con decisiones históricas.
- Registrar un historial de decisiones auditables para aprendizaje organizacional.
- Proveer un dashboard ejecutivo con visualización de tendencias, alertas y métricas de conocimiento.
- Garantizar la trazabilidad completa de todas las acciones sobre el sistema mediante un módulo de auditoría.

3. Alcance Técnico

3.1 Componentes del Sistema

Componente	Tecnología	Responsabilidad
Frontend Web	React 18, TypeScript, TailwindCSS, Recharts	Interfaz de usuario, dashboards, formularios
Backend API	Python 3.11, FastAPI, SQLAlchemy, Pydantic v2	Lógica de negocio, motor de reglas, APIs REST
Base de Datos	PostgreSQL 15, pgcrypto, pg_audit	Persistencia, integridad referencial, auditoría
Motor de Reglas	Python custom rule engine + JSON Schema	Evaluación de condiciones, inferencia de hechos
Autenticación	JWT (HS256), OAuth2 Password Flow	Gestión de sesiones, control de acceso
Infraestructura	Docker, Docker Compose, Nginx, Certbot	Contenedorización, proxy inverso, SSL
Monitoreo	Prometheus, Grafana, Sentry	Métricas, alertas, rastreo de errores

3.2 Límites del Sistema

El KBDSS no sustituye los sistemas ERP o CRM existentes en Nexus-Corp; se integra con ellos a través de APIs de intercambio de datos. El sistema no gestiona directamente inventarios físicos ni ejecuta transacciones financieras; su función es de apoyo a la decisión, no de ejecución transaccional.

3.3 Exclusiones

- Integración con hardware de almacén (WMS físico).
- Procesamiento de lenguaje natural (NLP) avanzado en la versión inicial.
- Integración con sistemas de terceros distintos a los definidos en la Fase 1.
- Módulo de inteligencia artificial generativa (previsto para Fase 3).

4. Definiciones, Acrónimos y Abreviaturas

Término / Acrónimo	Definición
KBDSS	Knowledge-Based Decision Support System. Sistema de soporte a la toma de decisiones basado en conocimiento.
KPI	Key Performance Indicator. Indicador Clave de Desempeño, métrica cuantificable de rendimiento organizacional.
RF	Requerimiento Funcional. Define el comportamiento específico que debe tener el sistema.
RNF	Requerimiento No Funcional. Define las características de calidad del sistema.
RN	Regla de Negocio. Condición o restricción que rige el comportamiento de los procesos organizacionales.
API	Application Programming Interface. Interfaz de Programación de Aplicaciones.
REST	Representational State Transfer. Estilo arquitectónico para servicios web.
JWT	JSON Web Token. Estándar para la transmisión segura de información entre partes.
RBAC	Role-Based Access Control. Control de Acceso Basado en Roles.
ORM	Object-Relational Mapping. Técnica de mapeo entre objetos y tablas relacionales.
DTD / DTO	Data Transfer Object. Objeto utilizado para transferir datos entre capas del sistema.
CI/CD	Continuous Integration / Continuous Deployment. Automatización del ciclo de vida del software.
Clean Architecture	Patrón arquitectónico de Robert C. Martin que separa responsabilidades en capas concéntricas.
What-If	Análisis de escenarios hipotéticos para evaluar el impacto de cambios en variables de decisión.
Motor de Reglas	Componente de software que evalúa condiciones lógicas definidas como reglas de conocimiento.
SGBD	Sistema Gestor de Base de Datos.

SSL/TLS	Secure Sockets Layer / Transport Layer Security. Protocolos de cifrado de comunicaciones.
CORS	Cross-Origin Resource Sharing. Mecanismo de control de acceso HTTP entre orígenes distintos.
UUID	Universally Unique Identifier. Identificador único universal de 128 bits.
JSON	JavaScript Object Notation. Formato ligero de intercambio de datos.
SPA	Single Page Application. Aplicación web de una sola página.
MVC	Model-View-Controller. Patrón de diseño de software.
UML	Unified Modeling Language. Lenguaje de modelado unificado para especificación de software.
IEEE 830	Estándar del Instituto de Ingenieros Eléctricos y Electrónicos para especificación de requisitos de software.
ER	Entidad-Relación. Modelo conceptual de base de datos.
DML	Data Manipulation Language. Lenguaje de manipulación de datos en SQL.
DDL	Data Definition Language. Lenguaje de definición de datos en SQL.
HTTP	HyperText Transfer Protocol. Protocolo de transferencia de hipertexto.
HTTPS	HTTP Secure. Versión segura del protocolo HTTP con cifrado TLS.
ERP	Enterprise Resource Planning. Sistema de planificación de recursos empresariales.

5. Referencias

- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC 25010:2011 – Systems and software Quality Requirements and Evaluation (SQuaRE).
- Clean Architecture: A Craftsman's Guide to Software Structure and Design – Robert C. Martin, 2017.
- FastAPI Documentation – Sebastián Ramírez. <https://fastapi.tiangolo.com>
- React Documentation – Meta Inc. <https://react.dev>
- PostgreSQL 15 Documentation – The PostgreSQL Global Development Group.
- OWASP Top 10:2021 – Open Web Application Security Project.
- RFC 7519 – JSON Web Token (JWT) – M. Jones, J. Bradley, N. Sakimura, 2015.
- Nexus-Corp: Arquitectura de Inteligencia Organizacional – Fase 1 – Documento Interno ETS-KBDSS-F1-2025.
- Docker Documentation – Docker Inc. <https://docs.docker.com>
- Prometheus Monitoring System – CNCF. <https://prometheus.io>
- UML 2.5.1 Specification – Object Management Group (OMG), 2017.

6. Descripción General del Producto

6.1 Perspectiva del Producto

El KBDSS se posiciona como un sistema de segunda generación en la evolución tecnológica de Nexus-Corp, complementando los sistemas transaccionales existentes (ERP, CRM, WMS) con capacidades analíticas y de gestión del conocimiento. El sistema opera en modo independiente pero se alimenta de datos exportados desde los sistemas core mediante APIs de integración.

Desde una perspectiva arquitectónica, el sistema sigue un patrón cliente-servidor con separación clara entre frontend (SPA en React), backend (FastAPI/Python) y capa de persistencia (PostgreSQL). La arquitectura interna del backend sigue los principios de Clean Architecture, garantizando independencia de frameworks, testabilidad y mantenibilidad a largo plazo.

6.2 Funciones Principales

#	Módulo	Descripción
M01	Gestión de Usuarios y Roles	Administración completa del ciclo de vida de usuarios, asignación de roles y permisos granulares basados en RBAC.
M02	Captura de Conocimiento	Interfaz para que expertos organizacionales registren reglas de negocio, condiciones y consecuencias en formato estructurado.
M03	Motor de Reglas	Núcleo computacional que evalúa reglas de conocimiento contra el estado actual del sistema y genera inferencias.
M04	Análisis What-If	Módulo de simulación que permite definir escenarios hipotéticos y evaluar su impacto mediante el motor de reglas.
M05	Gestión de KPIs	Definición, cálculo y monitoreo de indicadores clave de desempeño correlacionados con reglas y decisiones.
M06	Generación de Recomendaciones	Producción automática de recomendaciones accionables basadas en la evaluación de reglas y análisis de escenarios.
M07	Historial de Decisiones	Registro cronológico y auditado de todas las decisiones tomadas con o sin asistencia del sistema.
M08	Retroalimentación Organizacional	Captura del resultado real de las decisiones para ajuste y refinamiento de las reglas de conocimiento.
M09	Dashboard Ejecutivo	Visualización consolidada de KPIs, tendencias, alertas, estado del conocimiento y métricas de decisión.
M10	Auditoría y Trazabilidad	Registro inmutable de todos los cambios, accesos y operaciones realizadas sobre el sistema.

M11	Reportes y Exportación	Generación de reportes en PDF/Excel con filtros dinámicos sobre decisiones, KPIs y conocimiento.
M12	Administración General	Configuración global del sistema, gestión de parámetros, mantenimiento y respaldo.

6.3 Características de los Usuarios

Rol	Perfil Técnico	Responsabilidades en el Sistema
Administrador del Sistema	Alto – TI avanzado	Configuración global, gestión de usuarios, mantenimiento, respaldos, auditoría completa.
Administrador de Conocimiento	Medio-Alto – Analista de Negocio	Captura, validación y gestión del ciclo de vida de las reglas de conocimiento.
Decisor (Ejecutivo/Gerente)	Básico-Medio – Usuario de negocio	Consulta de dashboard, revisión de recomendaciones, aprobación de decisiones, retroalimentación.
Analista	Medio – Analista de datos	Ejecución de análisis What-If, gestión de escenarios, generación de reportes, revisión de KPIs.

6.4 Restricciones del Sistema

- El sistema debe operar en entornos Linux (Ubuntu 22.04 LTS o superior) para producción.
- El tiempo de respuesta de las APIs REST no debe superar los 2 segundos para operaciones estándar.
- La interfaz de usuario debe ser compatible con los navegadores Chrome 100+, Firefox 100+, Edge 100+.
- El sistema no almacenará información sensible de carácter personal sin mecanismos de cifrado en reposo.
- Todas las comunicaciones entre cliente y servidor se realizarán exclusivamente mediante HTTPS/TLS 1.3.
- La base de datos no podrá tener más de 3 conexiones simultáneas por usuario de base de datos.
- El sistema debe cumplir con las políticas internas de seguridad informática de Nexus-Corp v3.2.

6.5 Suposiciones y Dependencias

- Se asume que los servidores de producción disponen de un mínimo de 8 GB RAM, 4 vCPU y 100 GB SSD NVMe.
- Se asume conectividad a internet de alta disponibilidad (SLA ≥ 99.5%) para el entorno de producción.

- Se asume que el equipo de desarrollo dispone de acceso a registros de Docker Hub y repositorios NPM/PyPI.
- El proyecto depende de la disponibilidad de las APIs de los sistemas ERP/CRM de Nexus-Corp para integración futura.
- La disponibilidad de usuarios expertos de negocio para validación de reglas de conocimiento es crítica para las fases de captura.

7. Especificación IEEE 830 – Requerimientos Funcionales

Los requerimientos funcionales describen el comportamiento observable del sistema ante entradas específicas. Cada requerimiento está identificado con un código único, prioridad, descripción, entradas, salidas, precondiciones y postcondiciones.

RF-01: Gestión de Usuarios

RF-01 – Gestión de Usuarios	
Prioridad	ALTA
Descripción	El sistema debe permitir la creación, consulta, modificación, desactivación y eliminación lógica de usuarios del sistema. Cada usuario tiene un nombre de usuario único, correo electrónico verificado, contraseña cifrada (bcrypt), estado (activo/inactivo) y uno o más roles asignados.
Entradas	Datos del usuario (nombre, correo, contraseña, rol(es)); acciones CRUD del administrador.
Salidas / Resultados	Usuario creado/modificado/desactivado; notificación de bienvenida por correo; entrada en tabla de auditoría.
Precondiciones	El solicitante debe poseer rol 'Administrador del Sistema'.
Postcondiciones	El usuario queda registrado/actualizado en la base de datos con estado correcto y auditoría registrada.
Actor(es)	Administrador del Sistema
Reglas Relacionadas	RN-06, RN-07

RF-02: Gestión de Roles

RF-02 – Gestión de Roles	
Prioridad	ALTA
Descripción	El sistema debe implementar un modelo RBAC con cuatro roles base: Administrador del Sistema, Administrador de Conocimiento, Decisor y Analista. Cada rol tiene un conjunto de permisos predefinidos sobre módulos y operaciones. El administrador puede crear roles personalizados con permisos granulares.
Entradas	Nombre del rol, lista de permisos por módulo/operación.

Salidas / Resultados	Rol creado/modificado; permisos aplicados inmediatamente a los usuarios asignados.
Precondiciones	El solicitante debe ser Administrador del Sistema.
Postcondiciones	Los cambios de permisos se aplican en la siguiente solicitud autenticada de los usuarios afectados.
Actor(es)	Administrador del Sistema
Reglas Relacionadas	RN-06

RF-03: Gestión de Reglas de Conocimiento

RF-03 – Gestión de Reglas de Conocimiento	
Prioridad	CRÍTICA
Descripción	El sistema debe permitir crear, versionar, activar, desactivar, clonar y eliminar lógicamente reglas de conocimiento. Una regla consiste en: identificador, nombre, categoría, condición (expresión booleana estructurada), acción/recomendación resultante, peso/prioridad, y estado de validación. Las reglas deben soportar versionado semántico (MAJOR.MINOR.PATCH).
Entradas	Definición de regla: nombre, condición JSON, acción, categoría, peso, metadatos.
Salidas / Resultados	Regla creada/versión nueva; regla activada en motor; historial de versiones disponible.
Precondiciones	El usuario debe ser Administrador de Conocimiento o superior.
Postcondiciones	La regla queda almacenada con su versión, estado de validación y disponible para el motor de reglas si está activa.
Actor(es)	Administrador de Conocimiento
Reglas Relacionadas	RN-01 a RN-04

RF-04: Captura de Conocimiento

RF-04 – Captura de Conocimiento	
Prioridad	CRÍTICA

Descripción	El sistema debe proveer una interfaz asistida para que expertos organizacionales expresen conocimiento en lenguaje semi-natural, que el sistema transformará en estructuras de reglas. Incluye: formulario guiado de captura, sugerencias de autocompletado basadas en reglas existentes, previsualización de la regla antes de su guardado, y asistente de validación sintáctica en tiempo real.
Entradas	Texto de conocimiento del experto; campos de condición y acción; categoría; metadatos del dominio.
Salidas / Resultados	Borrador de regla generado y guardado; notificación al Administrador de Conocimiento para revisión.
Precondiciones	El usuario debe estar autenticado con rol Administrador de Conocimiento o Analista.
Postcondiciones	El borrador de conocimiento queda en estado 'Pendiente de Validación' en la base de datos.
Actor(es)	Administrador de Conocimiento, Analista
Reglas Relacionadas	RN-03, RN-04

RF-05: Validación de Conocimiento

RF-05 – Validación de Conocimiento	
Prioridad	ALTA
Descripción	El sistema debe implementar un flujo de validación de dos etapas para las reglas en estado borrador: (1) Validación sintáctica automática del motor de reglas; (2) Validación semántica por el Administrador de Conocimiento. Las reglas que no superen la validación sintáctica no podrán ser activadas. Las rechazadas en validación semántica requieren justificación escrita.
Entradas	Regla en estado borrador/pendiente; decisión del validador (aprobar/rechazar); comentarios.
Salidas / Resultados	Estado de la regla actualizado (Activa/Rechazada); notificación al creador; registro en auditoría.
Precondiciones	Existe una regla en estado 'Pendiente de Validación'.
Postcondiciones	La regla pasa a estado 'Activa' (integrada al motor) o 'Rechazada' (con justificación archivada).
Actor(es)	Administrador de Conocimiento

Reglas Relacionadas	RN-03
----------------------------	-------

RF-06: Gestión de Escenarios

RF-06 – Gestión de Escenarios	
Prioridad	ALTA
Descripción	El sistema debe permitir crear, guardar, nombrar, versionar y comparar escenarios de análisis. Un escenario es un conjunto de valores hipotéticos para variables de negocio (inventario, demanda, costos, tiempos) que sirven como entrada al motor de reglas para simulación What-If. Los escenarios pueden ser públicos (compartidos) o privados (del analista).
Entradas	Nombre del escenario; descripción; conjunto de variables con valores hipotéticos; alcance (público/privado).
Salidas / Resultados	Escenario guardado con identificador único; disponible para análisis What-If.
Precondiciones	El usuario debe ser Analista o superior.
Postcondiciones	El escenario queda almacenado y disponible para ejecución y comparación.
Actor(es)	Analista, Decisor
Reglas Relacionadas	RN-02, RN-05

RF-07: Análisis What-If

RF-07 – Análisis What-If	
Prioridad	CRÍTICA
Descripción	El sistema debe ejecutar análisis What-If aplicando el motor de reglas sobre uno o más escenarios, generando un reporte de impacto que muestre: reglas disparadas, recomendaciones activadas, KPIs proyectados y comparación entre escenarios. El análisis debe completarse en menos de 10 segundos para conjuntos de hasta 100 reglas activas.
Entradas	Escenario(s) seleccionado(s); conjunto de reglas activas; parámetros de ejecución.
Salidas / Resultados	Reporte de análisis: reglas disparadas, recomendaciones, KPIs proyectados, comparativa entre escenarios.

Precondiciones	Existen al menos un escenario y reglas activas en el sistema.
Postcondiciones	El reporte queda almacenado y vinculado al historial del analista; las recomendaciones pueden ser promovidas a decisiones.
Actor(es)	Analista, Decisor
Reglas Relacionadas	RN-01, RN-02, RN-05

RF-08: Generación de Recomendaciones

RF-08 – Generación de Recomendaciones	
Prioridad	CRÍTICA
Descripción	El motor de reglas debe generar automáticamente recomendaciones accionables cuando se cumplen las condiciones de las reglas activas. Cada recomendación incluye: descripción de la acción recomendada, nivel de confianza (basado en historial de efectividad), regla(s) que la originaron, impacto proyectado en KPIs, y plazo sugerido de implementación.
Entradas	Estado del sistema (variables de negocio); reglas activas; historial de efectividad.
Salidas / Resultados	Lista priorizada de recomendaciones con nivel de confianza y KPIs proyectados.
Precondiciones	El motor de reglas tiene al menos una regla activa y datos de entrada disponibles.
Postcondiciones	Las recomendaciones quedan registradas y disponibles para revisión y aprobación por Decisores.
Actor(es)	Sistema (automático), Analista, Decisor
Reglas Relacionadas	RN-04, RN-05

RF-09: Gestión de KPIs

RF-09 – Gestión de KPIs	
Prioridad	ALTA

Descripción	El sistema debe permitir definir, calcular y monitorear KPIs organizacionales. Cada KPI tiene: nombre, fórmula de cálculo, unidad de medida, valor objetivo (target), umbrales de alerta (warning/critical), frecuencia de actualización y categoría de negocio. El sistema calcula automáticamente los KPIs en los intervalos configurados y genera alertas cuando se superan umbrales.
Entradas	Definición del KPI (nombre, fórmula, target, umbrales); datos de entrada para cálculo.
Salidas / Resultados	Valor calculado del KPI; estado semáforo (verde/amarillo/rojo); alerta si corresponde.
Precondiciones	El usuario debe ser Administrador de Conocimiento o Analista.
Postcondiciones	El KPI queda registrado con su historial de valores y alertas generadas.
Actor(es)	Administrador de Conocimiento, Analista, Decisor
Reglas Relacionadas	RN-01, RN-02

RF-10: Retroalimentación Organizacional

RF-10 – Retroalimentación Organizacional	
Prioridad	ALTA
Descripción	El sistema debe proveer un módulo para registrar el resultado real de las decisiones tomadas con apoyo del KBDSS. La retroalimentación incluye: evaluación del resultado (positivo/negativo/neutral), desviación entre resultado proyectado y real, lecciones aprendidas, y sugerencias de ajuste a reglas. El sistema usa la retroalimentación para actualizar el nivel de confianza de las reglas.
Entradas	Decisión de referencia; evaluación del resultado; desviación cuantitativa; lecciones aprendidas.
Salidas / Resultados	Retroalimentación registrada; nivel de confianza de reglas actualizado; entrada en aprendizaje organizacional.
Precondiciones	Existe una decisión registrada en el historial que aún no tiene retroalimentación.
Postcondiciones	El nivel de confianza de las reglas involucradas se ajusta según la efectividad registrada.
Actor(es)	Decisor, Analista

Reglas Relacionadas	RN-05
----------------------------	-------

RF-11: Historial de Decisiones

RF-11 – Historial de Decisiones	
Prioridad	ALTA
Descripción	El sistema debe mantener un registro cronológico e inmutable de todas las decisiones tomadas, incluyendo: fecha/hora, decisor responsable, contexto de la decisión, escenario asociado, recomendaciones consideradas, decisión final adoptada, justificación y estado de implementación. El historial es consultable con filtros por fecha, usuario, categoría y estado.
Entradas	Datos de la decisión; referencias a escenario y recomendaciones; justificación del decisor.
Salidas / Resultados	Decisión registrada en el historial; disponible para consulta y retroalimentación posterior.
Precondiciones	El usuario debe ser Decisor o Administrador.
Postcondiciones	La decisión queda almacenada de forma inmutable en el historial.
Actor(es)	Decisor, Administrador del Sistema
Reglas Relacionadas	RN-06

RF-12: Auditoría de Cambios

RF-12 – Auditoría de Cambios	
Prioridad	ALTA
Descripción	El sistema debe registrar automáticamente en un log de auditoría toda operación de creación, modificación, eliminación o consulta sensible sobre cualquier entidad del sistema. Cada entrada del log incluye: timestamp UTC, usuario que realizó la acción, tipo de acción, entidad afectada, ID de registro, valores anteriores y nuevos (JSON diff), dirección IP y token de sesión.
Entradas	Toda operación CRUD del sistema; eventos de autenticación; cambios de configuración.

Salidas / Resultados	Entrada de auditoría creada automáticamente; inmutable, sin opción de borrado.
Precondiciones	El usuario debe estar autenticado.
Postcondiciones	Cada acción sensible queda registrada en la tabla de auditoría de forma atómica con la operación principal.
Actor(es)	Sistema (automático), Administrador del Sistema
Reglas Relacionadas	RN-06

RF-13: Reportes

RF-13 – Reportes	
Prioridad	MEDIA
Descripción	El sistema debe generar reportes dinámicos en formatos PDF y Excel sobre: decisiones históricas, KPIs por período, efectividad de reglas, análisis de escenarios comparativos y resúmenes de retroalimentación. Los reportes incluyen filtros por rango de fechas, usuario, categoría y estado. La generación de reportes debe completarse en menos de 30 segundos para conjuntos de hasta 10,000 registros.
Entradas	Parámetros de filtrado; tipo de reporte; formato de exportación.
Salidas / Resultados	Archivo PDF o Excel generado y disponible para descarga.
Precondiciones	El usuario debe tener rol Analista o superior.
Postcondiciones	El reporte queda disponible para descarga durante 24 horas antes de ser eliminado del almacenamiento temporal.
Actor(es)	Analista, Decisor, Administrador del Sistema
Reglas Relacionadas	N/A

RF-14: Dashboard Ejecutivo

RF-14 – Dashboard Ejecutivo	
Prioridad	ALTA

Descripción	El sistema debe proveer un dashboard personalizable con widgets de: resumen de KPIs (semáforo), gráficas de tendencias temporales, últimas recomendaciones activas, actividad reciente del sistema, estado de la base de conocimiento (total reglas activas/pendientes/rechazadas), y alertas críticas. El dashboard se actualiza en tiempo real o con frecuencia configurable (30s/1min/5min).
Entradas	Datos de KPIs, recomendaciones, actividad del sistema; configuración de widgets del usuario.
Salidas / Resultados	Dashboard renderizado con widgets actualizados; alertas destacadas.
Precondiciones	El usuario debe estar autenticado.
Postcondiciones	El dashboard muestra la información más reciente disponible según la frecuencia de actualización configurada.
Actor(es)	Decisor, Analista, Administrador del Sistema
Reglas Relacionadas	RN-01

RF-15: Administración General

RF-15 – Administración General	
Prioridad	ALTA
Descripción	El sistema debe proveer un panel de administración con funciones de: configuración de parámetros globales del sistema, gestión de conexiones con sistemas externos, visualización de métricas de rendimiento del servidor, gestión de respaldos de base de datos, control del motor de reglas (iniciar/detener/reiniciar), y gestión de notificaciones y alertas del sistema.
Entradas	Parámetros de configuración; comandos de administración.
Salidas / Resultados	Configuración aplicada; métricas de sistema actualizadas; respaldos ejecutados.
Precondiciones	Solo accesible para el rol Administrador del Sistema.
Postcondiciones	Los cambios de configuración se aplican inmediatamente o tras reinicio del servicio según corresponda.
Actor(es)	Administrador del Sistema
Reglas Relacionadas	RN-06, RN-07

8. Requerimientos No Funcionales

Los requerimientos no funcionales definen los atributos de calidad del sistema. Se clasifican siguiendo el modelo ISO/IEC 25010 adaptado a las necesidades del KBDSS.

ID	Categoría	Descripción	Criterio de Aceptación
RNF-01	Seguridad	Todas las contraseñas deben almacenarse usando bcrypt con factor de trabajo mínimo 12.	Verificación de hash en auditoría
RNF-02	Seguridad	Los tokens JWT deben tener tiempo de expiración máximo de 8 horas para sesiones activas.	Test de expiración de token
RNF-03	Seguridad	El sistema debe implementar rate limiting de 100 req/min por IP en endpoints públicos.	Test de carga con JMeter
RNF-04	Seguridad	Toda comunicación debe usar TLS 1.3; no se permite TLS 1.1 ni 1.2.	Escaneo SSL Labs A+
RNF-05	Seguridad	La API debe validar y sanitizar todas las entradas contra inyección SQL y XSS.	OWASP ZAP scan sin vulnerabilidades críticas
RNF-06	Rendimiento	Las APIs de consulta estándar deben responder en ≤ 500ms al percentil P95.	Test de carga con k6
RNF-07	Rendimiento	El análisis What-If con 100 reglas activas debe completarse en ≤ 10 segundos.	Prueba con dataset de referencia
RNF-08	Rendimiento	El dashboard ejecutivo debe renderizarse en ≤ 3 segundos con conexión de 10 Mbps.	Prueba Lighthouse score ≥ 85
RNF-09	Disponibilidad	El sistema debe mantener disponibilidad de ≥ 99.5% mensual (≤ 3.6 horas de downtime).	Monitoreo con Prometheus/Grafana
RNF-10	Disponibilidad	El sistema debe tener respaldo automático de BD cada 6 horas con retención de 30 días.	Verificación de backups restaurables
RNF-11	Escalabilidad	La arquitectura debe soportar escalar horizontalmente el backend añadiendo instancias sin modificación de código.	Despliegue multi-instancia funcional
RNF-12	Escalabilidad	La base de datos debe soportar hasta 100,000 reglas activas sin degradación de rendimiento.	Test con dataset sintético de 100K reglas

RNF-13	Usabilidad	El sistema debe ser operable por usuarios con conocimiento básico de herramientas digitales tras 2 horas de capacitación.	Prueba de usabilidad con usuarios reales
RNF-14	Usabilidad	La interfaz debe ser responsiva y operable en dispositivos con resolución mínima de 1280×768 px.	Test en resoluciones objetivo
RNF-15	Mantenibilidad	El código backend debe mantener cobertura de pruebas unitarias $\geq 80\%$.	Reporte de cobertura pytest
RNF-16	Mantenibilidad	La arquitectura Clean debe garantizar que cambios en la BD no afecten la capa de dominio.	Prueba de aislamiento de capas
RNF-17	Portabilidad	El sistema debe desplegarse mediante Docker Compose en cualquier host Linux sin configuración manual.	Despliegue en entorno limpio en < 15 min
RNF-18	Auditabilidad	El 100% de las operaciones CRUD sobre entidades críticas deben generar entrada en log de auditoría.	Verificación cruzada de registros
RNF-19	Compatibilidad	Las APIs REST deben cumplir con el estándar OpenAPI 3.1 y ser documentadas con Swagger UI.	Validación con swagger-cli
RNF-20	Confiabilidad	El motor de reglas debe producir resultados determinísticos y reproducibles para la misma entrada.	Test de idempotencia del motor

9. Reglas de Negocio

9.1 Reglas de Inventario

- RN-INV-01: Si el nivel de stock de un SKU cae por debajo del 20% del stock mínimo definido, el sistema debe generar automáticamente una recomendación de reabastecimiento urgente.
- RN-INV-02: Si la rotación de un producto es $< 5\%$ en 90 días, el sistema clasifica el artículo como inventario de baja rotación y recomienda revisión de precio o liquidación.
- RN-INV-03: El stock máximo de cualquier SKU no puede exceder el 150% de la demanda proyectada a 60 días para evitar sobrestock.
- RN-INV-04: Si un proveedor tiene más de 2 incumplimientos de entrega en 30 días, el sistema eleva su nivel de riesgo a ALTO y activa reglas de alerta de suministro.

9.2 Reglas de Demanda

- RN-DEM-01: Si la variación de la demanda semanal supera el $\pm 30\%$ respecto al promedio móvil de 4 semanas, el sistema activa una alerta de demanda anormal.
- RN-DEM-02: El sistema debe recalcular las proyecciones de demanda al menos una vez por semana con los datos reales más recientes.
- RN-DEM-03: Si la demanda proyectada supera el 90% de la capacidad de almacenamiento disponible, se activa una regla de contingencia de capacidad.

9.3 Reglas de Riesgo

- RN-RIE-01: Un proveedor es clasificado como riesgo CRÍTICO si tiene: tasa de incumplimiento $> 15\%$ O tiempo de entrega promedio $> 2x$ el SLA acordado.
- RN-RIE-02: Cualquier decisión con impacto financiero $> Q\ 500,000$ requiere aprobación de al menos 2 Decisores registrados en el sistema.
- RN-RIE-03: Las reglas de conocimiento con nivel de confianza $< 40\%$ son suspendidas automáticamente hasta revisión del Administrador de Conocimiento.

9.4 Reglas de Recomendaciones

- RN-REC-01: Las recomendaciones se priorizan por: $(\text{Impacto} * \text{Nivel_Confianza}) / \text{Urgencia}$, en orden descendente.
- RN-REC-02: Una recomendación no puede permanecer sin acción (aprobada/rechazada) por más de 72 horas; el sistema escala automáticamente al nivel jerárquico superior.
- RN-REC-03: Las recomendaciones generadas con nivel de confianza $< 60\%$ deben mostrarse con indicador de 'Requiere Revisión Manual'.

9.5 Reglas de Retroalimentación

- RN-RET-01: El nivel de confianza de una regla se actualiza mediante la fórmula:
$$NC_{\text{nuevo}} = NC_{\text{actual}} * 0.7 + \text{Efectividad_nueva} * 0.3.$$
- RN-RET-02: Si una regla recibe 3 retroalimentaciones negativas consecutivas, su estado cambia automáticamente a 'En Revisión'.
- RN-RET-03: La retroalimentación debe registrarse en un plazo máximo de 30 días después de la implementación de la decisión.

9.6 Reglas de Gestión del Conocimiento

- RN-CON-01: Toda regla de conocimiento debe pasar por el flujo de validación de dos etapas antes de ser activada en el motor de reglas.
- RN-CON-02: No pueden coexistir dos reglas activas con condiciones idénticas y categorías iguales; el sistema debe detectar y alertar sobre conflictos.
- RN-CON-03: El versionado de reglas es obligatorio; nunca se eliminan físicamente las versiones anteriores (solo se desactivan).
- RN-CON-04: Toda modificación a una regla activa genera automáticamente una nueva versión; la versión anterior se archiva automáticamente.

10. Modelado UML

10.1 Diagrama de Casos de Uso

El diagrama de casos de uso identifica las interacciones entre los actores del sistema y las funcionalidades ofrecidas. Los actores principales son: Administrador del Sistema (AS), Administrador de Conocimiento (AC), Decisor (DE) y Analista (AN).

10.1.1 Código Mermaid – Diagrama de Casos de Uso

```
%%{init: {'theme': 'default'}}%%
graph TD
    AS([Administrador del Sistema])
    AC([Administrador de Conocimiento])
    DE([Decisor])
    AN([Analista])

    AS --> UC01[UC-01 Gestionar Usuarios]
    AS --> UC02[UC-02 Gestionar Roles]
    AS --> UC15[UC-15 Administración General]
    AS --> UC12[UC-12 Auditar Cambios]
    AS --> UC11[UC-11 Consultar Historial Decisiones]

    AC --> UC03[UC-03 Gestionar Reglas de Conocimiento]
    AC --> UC04[UC-04 Capturar Conocimiento]
    AC --> UC05[UC-05 Validar Conocimiento]
    AC --> UC09[UC-09 Gestionar KPIs]

    AN --> UC04
    AN --> UC06[UC-06 Gestionar Escenarios]
    AN --> UC07[UC-07 Ejecutar Análisis What-If]
    AN --> UC09
    AN --> UC13[UC-13 Generar Reportes]

    DE --> UC07
    DE --> UC08[UC-08 Revisar Recomendaciones]
    DE --> UC10[UC-10 Registrar Retroalimentación]
    DE --> UC11
    DE --> UC14[UC-14 Consultar Dashboard Ejecutivo]

    UC07 -.->|include| UC08
    UC04 -.->|include| UC05
    UC03 -.->|include| UC12
    UC08 -.->|extend| UC11
```

10.2 Especificación Textual de Casos de Uso

UC-01: Gestionar Usuarios

Objetivo	Permitir al Administrador del Sistema crear, modificar, desactivar y eliminar lógicamente usuarios del KBDSS.
Actores	Administrador del Sistema (primario)
Precondiciones	El actor debe estar autenticado con rol Administrador del Sistema.
Flujo Principal	1. El administrador accede al módulo de Gestión de Usuarios. 2. Selecciona la acción (Nuevo/Editar/Desactivar). 3. Completa/modifica el formulario de datos del usuario. 4. El sistema valida los datos (unicidad de correo/username, formato). 5. El sistema guarda los cambios y registra la auditoría. 6. El sistema envía notificación al usuario afectado.
Flujos Alternativos	FA-01: Correo ya registrado → el sistema muestra error de unicidad. FA-02: Correo no válido → validación en frontend rechaza el formato. FA-03: Intento de eliminar al último administrador → el sistema rechaza con mensaje de error.
Postcondiciones	El usuario queda creado/actualizado/desactivado en la base de datos con registro de auditoría.

UC-03: Gestionar Reglas de Conocimiento

Objetivo	Permitir al Administrador de Conocimiento gestionar el ciclo de vida completo de las reglas de conocimiento.
Actores	Administrador de Conocimiento (primario)
Precondiciones	Autenticación con rol Administrador de Conocimiento. La categoría de la regla debe existir.
Flujo Principal	1. El AC accede al módulo de Gestión de Reglas. 2. Selecciona Crear Nueva Regla. 3. Define nombre, categoría, condición (editor JSON), acción, peso y metadatos. 4. El sistema ejecuta validación sintáctica automática. 5. Si válida, la regla se guarda en estado Pendiente. 6. El AC revisa y activa/rechaza la regla en el flujo de validación.
Flujos Alternativos	FA-01: Error sintáctico en la condición → el editor muestra el error en línea. FA-02: Conflicto con regla existente → el sistema alerta sobre la duplicidad.
Postcondiciones	La regla queda en estado Activa (si fue aprobada) o Rechazada (con justificación), con versión asignada.

UC-07: Ejecutar Análisis What-If

Objetivo	Ejecutar simulación de escenarios hipotéticos para evaluar el impacto de variables de negocio.
----------	--

Actores	Analista (primario), Decisor (secundario)
Precondiciones	Existen reglas activas en el motor. Existe al menos un escenario guardado o se define uno nuevo.
Flujo Principal	1. El Analista accede al módulo What-If. 2. Selecciona o crea un escenario con variables hipotéticas. 3. Inicia la ejecución del análisis. 4. El motor de reglas evalúa todas las reglas activas contra el escenario. 5. El sistema genera el reporte de impacto (reglas disparadas, recomendaciones, KPIs proyectados). 6. El Analista revisa y puede comparar con otros escenarios. 7. Puede promover las recomendaciones a decisiones.
Flujos Alternativos	FA-01: Timeout > 10s → el sistema muestra error de rendimiento y permite reintentar. FA-02: Sin reglas activas → el sistema informa la necesidad de activar reglas primero.
Postcondiciones	El reporte de análisis queda almacenado. Las recomendaciones están disponibles para aprobación.

UC-10: Registrar Retroalimentación

Objetivo	Registrar el resultado real de una decisión para ajustar el nivel de confianza de las reglas involucradas.
Actores	Decisor (primario), Analista (secundario)
Precondiciones	Existe una decisión registrada sin retroalimentación y cuya fecha de implementación no excede 30 días.
Flujo Principal	1. El Decisor accede al historial de decisiones. 2. Selecciona la decisión a retroalimentar. 3. Registra: resultado (positivo/negativo/neutral), desviación cuantitativa, lecciones aprendidas. 4. El sistema actualiza el nivel de confianza de las reglas involucradas. 5. Se registra la retroalimentación en la base de datos.
Flujos Alternativos	FA-01: Decisión ya retroalimentada → el sistema muestra aviso y permite agregar comentario adicional.
Postcondiciones	Retroalimentación registrada. Niveles de confianza de reglas actualizados según RN-RET-01.

10.3 Diagrama de Clases

El diagrama de clases modela las entidades principales del dominio del KBDSS y sus relaciones. Se modela en notación UML 2.5.

10.3.1 Código Mermaid – Diagrama de Clases

```
classDiagram
class Usuario {
+UUID id
+String nombre_usuario
+String correo
+String password_hash
+Boolean activo
+DateTime creado_en
+DateTime ultimo_login
+autenticar(pwd) Boolean
+cambiarPassword(nuevo) void
}

class Rol {
+UUID id
+String nombre
+String descripcion
+List~Permiso~ permisos
+agregarPermiso(p) void
+removePermiso(p) void
}

class Regla {
+UUID id
+String nombre
+String categoria
+JSON condicion
+String accion
+Float peso
+Float nivel_confianza
+String version
+String estado
+validarSintaxis() Boolean
+versionar() Regla
+calcularConfianza() Float
}

class Conocimiento {
+UUID id
+String titulo
+String descripcion
+String dominio
+String estado_validacion
+UUID creado_por
+DateTime capturado_en
+validar() Boolean
+promoverARegla() Regla
}
```

```
class Escenario {
    +UUID id
    +String nombre
    +String descripcion
    +JSON variables
    +Boolean es_publico
    +UUID creado_por
    +ejecutarAnalisis() ResultadoAnalisis
    +clonar() Escenario
}

class Decision {
    +UUID id
    +String titulo
    +String descripcion
    +String estado
    +Float impacto_financiero
    +DateTime fecha_decision
    +UUID decisor_id
    +registrar() void
    +aprobar() void
}

class Recomendacion {
    +UUID id
    +String descripcion
    +Float nivel_confianza
    +Float impacto_proyectado
    +String prioridad
    +String estado
    +List~UUID~ reglas_origen
    +calcularPrioridad() Float
    +aprobar() void
}

class KPI {
    +UUID id
    +String nombre
    +String formula
    +Float valor_actual
    +Float target
    +Float umbral_warning
    +Float umbral_critical
    +String unidad
    +calcular() Float
    +obtenerEstado() String
    +generarAlerta() Alerta
}

class Retroalimentacion {
    +UUID id
    +UUID decision_id
    +String resultado
    +Float desviacion
    +String lecciones_aprendidas
```

```
+DateTime registrado_en  
+ajustarConfianza() void  
}
```

```
class Auditoria {  
+UUID id  
+String accion  
+String entidad  
+UUID entidad_id  
+JSON valores_anteriores  
+JSON valores_nuevos  
+String ip_origen  
+UUID usuario_id  
+DateTime timestamp  
}
```

```
Usuario "*" -- "*" Rol : tiene  
Conocimiento "1" -- "0..*" Regla : genera  
Regla "*" -- "0..*" Recomendacion : origina  
Escenario "1" -- "0..*" Recomendacion : produce  
Decision "1" -- "0..1" Retroalimentacion : recibe  
Decision "1" -- "0..*" Recomendacion : considera  
Recomendacion "*" -- "1..*" KPI : afecta  
Usuario "1" -- "*" Auditoria : genera  
Regla "1" -- "*" Auditoria : registra
```

10.4 Diagramas de Secuencia

10.4.1 Secuencia: Captura de Conocimiento

```
sequenceDiagram
    actor AC as Administrador de Conocimiento
    participant FE as Frontend React
    participant API as FastAPI Backend
    participant SVC as KnowledgeService
    participant DB as PostgreSQL
    participant AUD as AuditoriaService

    AC->>FE: Accede a módulo Captura de Conocimiento
    FE->>API: GET /api/v1/knowledge/categories
    API-->>FE: Lista de categorías
    AC->>FE: Completa formulario de conocimiento
    FE->>FE: Validación frontend (Zod)
    FE->>API: POST /api/v1/knowledge/
    API->>SVC: capturarConocimiento(dto)
    SVC->>SVC: validarSintaxisRegla(dto.condicion)
    alt Sintaxis válida
        SVC->>DB: INSERT INTO conocimientos VALUES(...)
        DB-->>SVC: conocimiento_id
        SVC->>AUD: registrar(CREAR, conocimiento_id, usuario)
        AUD->>DB: INSERT INTO auditorias VALUES(...)
        SVC-->>API: ConocimientoDTO (estado=PENDIENTE)
        API-->>FE: 201 Created + ConocimientoDTO
        FE-->>AC: Éxito - Conocimiento pendiente de validación
    else Sintaxis inválida
        SVC-->>API: ValidationError + detalle
        API-->>FE: 422 Unprocessable Entity
        FE-->>AC: Error - descripción del problema
    end
end
```

10.4.2 Secuencia: Ejecución de Análisis What-If

```
sequenceDiagram
    actor AN as Analista
    participant FE as Frontend React
    participant API as FastAPI Backend
    participant ASVC as AnalysisService
    participant RE as RuleEngine
    participant DB as PostgreSQL

    AN->>FE: Selecciona escenario y ejecuta análisis
    FE->>API: POST /api/v1/analysis/whatif
    API->>ASVC: ejecutarWhatIf(escenario_id, parametros)
    ASVC->>DB: SELECT * FROM reglas WHERE estado='ACTIVA'
    DB-->>ASVC: Lista de reglas activas
    ASVC->>DB: SELECT * FROM escenarios WHERE id=escenario_id
    DB-->>ASVC: Variables del escenario
    ASVC->>RE: evaluar(reglas, variables_escenario)
    loop Para cada regla activa
        RE->>RE: evaluarCondicion(regla.condicion, variables)
        alt Condición cumplida
            RE->>RE: agregarRecomendacion(regla.accion)
        end
    end
```

```

    end
  end
  RE-->>ASVC: ResultadoEvaluacion(reglas_disparadas, recomendaciones)
  ASVC-->>ASVC: calcularKPIsProyectados(resultado)
  ASVC-->>ASVC: priorizarRecomendaciones(recomendaciones)
  ASVC-->>DB: INSERT INTO analisis_resultados VALUES(...)
  ASVC-->>API: ReporteWhatIfDTO
  API-->>FE: 200 OK + ReporteWhatIfDTO
  FE-->>AN: Muestra reporte con gráficas y recomendaciones

```

10.4.3 Secuencia: Generación de Recomendación

```

sequenceDiagram
    participant RE as RuleEngine
    participant RSVC as RecomendacionService
    participant KSVC as KPIService
    participant DB as PostgreSQL
    participant NOTIF as NotificationService
    actor DE as Decisor

    RE->>RSVC: crearRecomendacion(regla_id, accion, variables)
    RSVC->>RSVC: calcularNivelConfianza(regla.nivel_confianza, historial)
    RSVC->>KSVC: proyectarImpactoKPIs(accion, variables)
    KSVC-->>RSVC: impacto_proyectado_kpis
    RSVC->>RSVC: calcularPrioridad(impacto, confianza, urgencia)
    RSVC->>DB: INSERT INTO recomendaciones VALUES(...)
    DB-->>RSVC: recomendacion_id
    alt Nivel confianza < 60%
        RSVC->>RSVC: marcar(requiere_revision_manual=true)
    end
    RSVC->>NOTIF: notificarDecisores(recomendacion_id)
    NOTIF->>DB: SELECT usuarios WHERE rol='DECISOR'
    NOTIF-->>DE: Notificación push/email: nueva recomendación
    RSVC-->>RE: RecomendacionDTO{id, descripcion, prioridad, confianza}

```

10.4.4 Secuencia: Registro de Retroalimentación

```

sequenceDiagram
    actor DE as Decisor
    participant FE as Frontend React
    participant API as FastAPI Backend
    participant FBSVC as RetroalimentacionService
    participant RE as RuleEngine
    participant DB as PostgreSQL

    DE->>FE: Accede a historial y selecciona decisión
    FE->>API: GET /api/v1/decisions/{id}
    API-->>FE: DecisionDTO con recomendaciones vinculadas
    DE->>FE: Completa formulario de retroalimentación
    FE->>API: POST /api/v1/feedback/
    API->>FBSVC: registrarRetroalimentacion(dto)
    FBSVC->>DB: INSERT INTO retroalimentaciones VALUES(...)
    FBSVC->>DB: SELECT reglas vinculadas a la decisión
    DB-->>FBSVC: Lista de reglas_id
    loop Para cada regla vinculada

```

```
FBSVC->>RE: actualizarConfianza(regla_id, efectividad)
RE->>RE: NC = NC*0.7 + efectividad*0.3
RE->>DB: UPDATE reglas SET nivel_confianza=NC WHERE id=regla_id
alt NC < 0.4
  RE->>DB: UPDATE reglas SET estado='SUSPENSION' WHERE id=regla_id
end
end
FBSVC-->>API: RetroalimentacionDTO
API-->>FE: 201 Created
FE-->>DE: Retroalimentación registrada correctamente
```

10.5 Diagramas de Actividades

10.5.1 Actividad: Flujo de Validación de Reglas

```
stateDiagram-v2
    [*] --> CapturaIniciada : AC crea borrador
    CapturaIniciada --> ValidacionSintactica : Sistema valida
    ValidacionSintactica --> ErrorSintactico : Condición inválida
    ErrorSintactico --> CapturaIniciada : AC corrige
    ValidacionSintactica --> PendienteRevision : Sintaxis OK
    PendienteRevision --> EnRevision : AC asigna revisor
    EnRevision --> Aprobada : Revisor aprueba
    EnRevision --> Rechazada : Revisor rechaza (con justificación)
    Rechazada --> CapturaIniciada : AC revisa y corrige
    Aprobada --> Activa : Motor integra regla
    Activa --> EnRevision : 3 retroalimentaciones negativas
    Activa --> Suspendida : NC < 40%
    Suspendida --> EnRevision : AC reactiva revisión
    Activa --> Archivada : Nueva versión creada
    Archivada --> [*]
```

10.5.2 Actividad: Proceso de Toma de Decisión Asistida

```
flowchart TD
    A([Inicio: Necesidad de Decisión]) --> B[Analista define variables del contexto]
    B --> C[Seleccionar/Crear Escenario]
    C --> D[Ejecutar Análisis What-If]
    D --> E{Motor de Reglas}
    E --> F[Reglas disparadas]
    F --> G[Generar Recomendaciones priorizadas]
    G --> H{Sin reglas aplicables}
    H --> I[Alerta: Base de conocimiento insuficiente]
    I --> J[AC revisa y agrega reglas]
    J --> D
    F --> K[Decisor revisa recomendaciones]
    K --> L{Nivel de confianza}
    L --> M{>= 60%}
    M --> N[Aprobar recomendación como decisión]
    L --> O{< 60%}
    O --> P[Solicitar revisión manual adicional]
    P --> Q[Analista realiza análisis complementario]
    Q --> L
    N --> R[Registrar Decisión en Historial]
    R --> S[Implementar Decisión operacionalmente]
    S --> T{Resultado en 30 días}
    T --> U{Positivo}
    T --> V{Negativo}
    T --> W{Neutral}
    U --> X[Retroalimentación positiva]
    V --> Y[Retroalimentación negativa + lecciones]
    W --> Z[Retroalimentación neutral]
    X & Y & Z --> AA[Actualizar nivel de confianza de reglas]
    AA --> AB([Fin: Ciclo de aprendizaje completado])
```

10.6 Diagrama de Componentes

```
graph TB
    subgraph CLIENT["Cliente (Navegador Web)"]
        UI_AUTH[Módulo Autenticación]
        UI_DASH[Dashboard Ejecutivo]
        UI_KNOW[Gestión Conocimiento]
        UI_WHATIF[Análisis What-If]
        UI_FEED[Retroalimentación]
        UI_ADMIN[Administración]
        HTTP_CLIENT[Axios HTTP Client]
        AUTH_CTX[Auth Context / JWT Store]
    end

    subgraph NGINX["Nginx Reverse Proxy / SSL Termination"]
        STATIC[Static File Server]
        PROXY[API Proxy /api/v1/]
    end

    subgraph BACKEND["FastAPI Backend"]
        ROUTER[API Router Layer]
        AUTH_MW[JWT Middleware / RBAC]
        subgraph SERVICES["Application Services"]
            USER_SVC[UserService]
            KNOW_SVC[KnowledgeService]
            RULE_SVC[RuleService]
            ANALYSIS_SVC[AnalysisService]
            DECISION_SVC[DecisionService]
            KPI_SVC[KPIService]
            FEEDBACK_SVC[FeedbackService]
            REPORT_SVC[ReportService]
            AUDIT_SVC[AuditService]
        end
        RULE_ENGINE[Motor de Reglas]
        ORM[SQLAlchemy ORM]
    end

    subgraph DATA["Capa de Datos"]
        POSTGRES[(PostgreSQL 15)]
        REDIS[(Redis Cache)]
        FILE_STORE[/Almacenamiento Reportes/]
    end

    CLIENT -->|HTTPS| NGINX
    NGINX -->|Forward /api| ROUTER
    ROUTER --> AUTH_MW
    AUTH_MW --> SERVICES
    SERVICES --> RULE_ENGINE
    SERVICES --> ORM
    ORM --> POSTGRES
    ANALYSIS_SVC --> REDIS
    REPORT_SVC --> FILE_STORE
```

10.7 Diagrama de Despliegue

```
graph TB
    subgraph INTERNET["Internet / Cliente"]
        BROWSER[Navegador del Usuario]
    end

    subgraph VPS["Servidor VPS / Cloud (Ubuntu 22.04)"]
        direction TB
        subgraph DOCKER["Docker Engine"]
            NGINX_C["Contenedor: Nginx:alpine\nPort: 80/443"]
            REACT_BUILD["Build React (dist/)"]
            FASTAPI_C["Contenedor: FastAPI\nPort: 8000 (interno)"]
            POSTGRES_C["Contenedor: PostgreSQL 15\nPort: 5432 (interno)"]
            REDIS_C["Contenedor: Redis 7\nPort: 6379 (interno)"]
            PROMETHEUS_C["Contenedor: Prometheus\nPort: 9090"]
            GRAFANA_C["Contenedor: Grafana\nPort: 3000"]
        end
        CERTBOT[Certbot SSL / Lets Encrypt]
        VOLUME_DB[(Volumen: pg_data)]
        VOLUME_REPORTS[(Volumen: reports_storage)]
    end

    BROWSER -->|HTTPS 443| NGINX_C
    NGINX_C -->|Serve| REACT_BUILD
    NGINX_C -->|Proxy /api| FASTAPI_C
    FASTAPI_C -->|SQL| POSTGRES_C
    FASTAPI_C -->|Cache| REDIS_C
    FASTAPI_C -->|Write| VOLUME_REPORTS
    POSTGRES_C --- VOLUME_DB
    PROMETHEUS_C -->|Scrape| FASTAPI_C
    GRAFANA_C -->|Query| PROMETHEUS_C
    CERTBOT --> NGINX_C
```

11. Arquitectura del Sistema

11.1 Arquitectura General de Software

El KBDSS adopta una arquitectura en tres niveles (Three-Tier Architecture) con separación clara entre presentación, lógica de negocio y datos, combinada con los principios de Clean Architecture para el backend y una arquitectura de componentes reactivos para el frontend.

La arquitectura garantiza: independencia de frameworks (el dominio no depende de FastAPI ni de React), testabilidad (cada capa puede probarse en aislamiento), mantenibilidad (bajo acoplamiento, alta cohesión) y escalabilidad (cada servicio puede escalar independientemente).

11.2 Clean Architecture – Capas del Sistema

11.2.1 Diagrama de Capas

```
graph TD
    subgraph DOMAIN["DOMAIN LAYER (Núcleo)"]
        ENTITIES["Entidades de Dominio\n(Usuario, Regla, Escenario, Decision...)"]
        VALUE_OBJ["Value Objects\n(NivelConfianza, EstadoRegla)"]
        DOMAIN_RULES["Reglas de Dominio\n(invariantes, validaciones)"]
        DOM_REPO["Interfaces de Repositorio\n(IREglaRepo, IDecisionRepo)"]
    end
    subgraph APPLICATION["APPLICATION LAYER"]
        USE_CASES["Casos de Uso\n(CapturaConocimientoUC, WhatIfUC)"]
        APP_DTO["DTOs de Aplicación"]
        APP_PORTS["Puertos (Interfaces de Servicio)"]
    end
    subgraph INFRA["INFRASTRUCTURE LAYER"]
        ORM_MODELS["Modelos SQLAlchemy"]
        REPO_IMPL["Implementaciones de Repositorio"]
        EXTERNAL["Servicios Externos (Email, Cache)"]
        MIGRATION["Alembic Migrations"]
    end
    subgraph PRESENTATION["PRESENTATION LAYER"]
        FASTAPI_ROUTES["FastAPI Routers"]
        REQUEST_SCHEMAS["Pydantic Schemas (Request/Response)"]
        MIDDLEWARE["Middleware (Auth, CORS, Logging)"]
    end
    PRESENTATION --> APPLICATION
    APPLICATION --> DOMAIN
    INFRA --> DOMAIN
    PRESENTATION -.->|No directo| INFRA
```

11.2.2 Capa de Dominio (Domain Layer)

La capa de dominio es el núcleo del sistema y no tiene dependencias externas. Contiene las entidades de negocio con sus invariantes y las interfaces de repositorio (puertos de salida). Las entidades de dominio no son modelos de base de datos; son objetos ricos con comportamiento propio. Por ejemplo, la entidad Regla encapsula la lógica de versionado semántico y el cálculo de nivel de confianza.

```
# domain/entities/regla.py
```

```

from dataclasses import dataclass
from typing import Optional
from uuid import UUID, uuid4
from enum import Enum

class EstadoRegla(Enum):
    BORRADOR = 'BORRADOR'
    PENDIENTE = 'PENDIENTE'
    ACTIVA = 'ACTIVA'
    RECHAZADA = 'RECHAZADA'
    SUSPENDIDA = 'SUSPENDIDA'
    ARCHIVADA = 'ARCHIVADA'

@dataclass
class Regla:
    id: UUID
    nombre: str
    condicion: dict
    accion: str
    nivel_confianza: float
    estado: EstadoRegla
    version: str # SemVer: 1.0.0

    def versionar(self) -> 'Regla':
        major, minor, patch = map(int, self.version.split('.'))
        nueva = Regla(id=uuid4(), nombre=self.nombre,
                      condicion=self.condicion, accion=self.accion,
                      nivel_confianza=self.nivel_confianza,
                      estado=EstadoRegla.BORRADOR,
                      version=f'{major}.{minor+1}.0')
        self.estado = EstadoRegla.ARCHIVADA
        return nueva

    def actualizar_confianza(self, efectividad: float) -> None:
        self.nivel_confianza = self.nivel_confianza * 0.7 + efectividad * 0.3
        if self.nivel_confianza < 0.4:
            self.estado = EstadoRegla.SUSPENDIDA

```

11.2.3 Capa de Aplicación (Application Layer)

La capa de aplicación orquesta los casos de uso del sistema. Cada caso de uso es una clase independiente con un único método `execute()` que recibe un DTO de entrada y retorna un DTO de salida. Esta capa depende únicamente de las interfaces definidas en el dominio, nunca de implementaciones concretas.

```

# application/use_cases/captura_conocimiento.py
from dataclasses import dataclass
from domain.repositories.i_conocimiento_repo import IConocimientoRepo
from domain.repositories.i_auditoria_repo import IAuditoriaRepo
from domain.entities.conocimiento import Conocimiento, EstadoValidacion

@dataclass
class CapturaConocimientoInput:
    titulo: str
    descripcion: str
    condicion_json: dict

```

```

    accion: str
    categoria: str
    usuario_id: str

class CapturaConocimientoUC:
    def __init__(self, repo: IConocimientoRepo, audit: IAuditoriaRepo):
        self._repo = repo
        self._audit = audit

    def execute(self, inp: CapturaConocimientoInput) -> Conocimiento:
        conocimiento = Conocimiento(
            titulo=inp.titulo, descripcion=inp.descripcion,
            condicion=inp.condicion_json, accion=inp.accion,
            categoria=inp.categoria, creado_por=inp.usuario_id,
            estado_validacion=EstadoValidacion.PENDIENTE
        )
        saved = self._repo.guardar(conocimiento)
        self._audit.registrar('CREAR', 'conocimiento', str(saved.id),
                               inp.usuario_id)
        return saved

```

11.2.4 Capa de Infraestructura (Infrastructure Layer)

La capa de infraestructura implementa las interfaces definidas en el dominio utilizando tecnologías concretas: SQLAlchemy para la persistencia, Redis para caché, y clientes HTTP para integraciones externas. Esta capa contiene los modelos ORM, las migraciones Alembic y las implementaciones de repositorio.

11.2.5 Capa de Presentación (Presentation Layer)

La capa de presentación expone las funcionalidades del sistema a través de la API REST de FastAPI. Contiene los routers, los schemas de validación Pydantic y los middlewares de autenticación, CORS y logging. Esta capa no contiene lógica de negocio; únicamente traduce las peticiones HTTP en llamadas a los casos de uso y formatea las respuestas.

11.3 Arquitectura del Motor de Reglas

El motor de reglas es el componente central del KBDSS. Implementa un motor de encadenamiento hacia adelante (forward chaining) basado en el algoritmo RETE simplificado, optimizado para el conjunto de reglas esperado (< 10,000 reglas activas).

```

# infrastructure/rule_engine/engine.py
from typing import List, Dict, Any
from domain.entities.regla import Regla
import operator

OPERATORS = {'==': operator.eq, '!=': operator.ne,
             '>': operator.gt, '>=': operator.ge,
             '<': operator.lt, '<=': operator.le}

class RuleEngine:
    def __init__(self, reglas: List[Regla]):
        self.reglas = sorted(reglas, key=lambda r: r.peso, reverse=True)

    def evaluar(self, hechos: Dict[str, Any]) -> List[dict]:

```

```
recomendaciones = []
for regla in self.reglas:
    if self._evaluar_condicion(regla.condicion, hechos):
        recomendaciones.append({
            'regla_id': str(regla.id),
            'accion': regla.accion,
            'confianza': regla.nivel_confianza,
            'peso': regla.peso
        })
return recomendaciones

def _evaluar_condicion(self, condicion: dict, hechos: dict) -> bool:
    # condicion: {campo: str, operador: str, valor: Any}
    campo = condicion.get('campo')
    op = OPERATORS.get(condicion.get('operador'))
    valor = condicion.get('valor')
    if not (campo and op):
        return False
    hecho_valor = hechos.get(campo)
    if hecho_valor is None:
        return False
    return op(hecho_valor, valor)
```

12. Diseño de Base de Datos

12.1 Modelo Conceptual

El modelo conceptual del KBDSS identifica las entidades principales y sus relaciones semánticas. Las entidades centrales son: Usuario, Rol, Conocimiento, Regla, Escenario, Decision, Recomendacion, KPI, Retroalimentacion y Auditoria.

12.2 Diagrama Entidad-Relación

```
erDiagram
    USUARIOS {
        UUID id PK
        VARCHAR nombre_usuario UK
        VARCHAR correo UK
        VARCHAR password_hash
        BOOLEAN activo
        TIMESTAMP creado_en
        TIMESTAMP ultimo_login
    }
    ROLES {
        UUID id PK
        VARCHAR nombre UK
        TEXT descripcion
        BOOLEAN activo
    }
    USUARIOS_ROLES {
        UUID usuario_id FK
        UUID rol_id FK
        TIMESTAMP asignado_en
    }
    CONOCIMIENTOS {
        UUID id PK
        VARCHAR titulo
        TEXT descripcion
        JSONB condicion
        TEXT accion
        VARCHAR categoria
        VARCHAR estado_validacion
        UUID creado_por FK
        TIMESTAMP creado_en
    }
    REGLAS {
        UUID id PK
        VARCHAR nombre
        VARCHAR categoria
        JSONB condicion
        TEXT accion
        FLOAT peso
        FLOAT nivel_confianza
        VARCHAR version
        VARCHAR estado
        UUID conocimiento_id FK
        UUID creado_por FK
        TIMESTAMP creado_en
    }
```

```
}
ESCENARIOS {
  UUID id PK
  VARCHAR nombre
  TEXT descripcion
  JSONB variables
  BOOLEAN es_publico
  UUID creado_por FK
  TIMESTAMP creado_en
}
DECISIONES {
  UUID id PK
  VARCHAR titulo
  TEXT descripcion
  VARCHAR estado
  FLOAT impacto_financiero
  UUID escenario_id FK
  UUID decisor_id FK
  TIMESTAMP fecha_decision
}
RECOMENDACIONES {
  UUID id PK
  TEXT descripcion
  FLOAT nivel_confianza
  FLOAT impacto_proyectado
  VARCHAR prioridad
  VARCHAR estado
  UUID escenario_id FK
  TIMESTAMP generada_en
}
KPIS {
  UUID id PK
  VARCHAR nombre
  TEXT formula
  FLOAT valor_actual
  FLOAT target
  FLOAT umbral_warning
  FLOAT umbral_critical
  VARCHAR unidad
  VARCHAR estado_semaforo
  TIMESTAMP calculado_en
}
RETROALIMENTACIONES {
  UUID id PK
  UUID decision_id FK
  VARCHAR resultado
  FLOAT desviacion
  TEXT lecciones_aprendidas
  UUID registrado_por FK
  TIMESTAMP registrado_en
}
AUDITORIAS {
  UUID id PK
  VARCHAR accion
  VARCHAR entidad
  UUID entidad_id
  JSONB valores_anteriores
}
```

```
JSONB valores_nuevos
VARCHAR ip_origen
UUID usuario_id FK
TIMESTAMP timestamp_utc
}

USUARIOS ||--o{ USUARIOS_ROLES : tiene
ROLES ||--o{ USUARIOS_ROLES : asignado_a
USUARIOS ||--o{ CONOCIMIENTOS : crea
CONOCIMIENTOS ||--o{ REGLAS : genera
REGLAS ||--o{ RECOMENDACIONES : origina
ESCENARIOS ||--o{ DECISIONES : contextualiza
ESCENARIOS ||--o{ RECOMENDACIONES : produce
DECISIONES ||--o{ RETROALIMENTACIONES : recibe
USUARIOS ||--o{ AUDITORIAS : genera
```

12.3 Diccionario de Datos – Tablas Principales

Tabla: usuarios

Campo	Tipo	Nulo	Restricción	Descripción
id	UUID	NO	PK, DEFAULT gen_random_uuid()	Identificador único del usuario
nombre_usuario	VARCHAR(50)	NO	UNIQUE, NOT NULL	Nombre de usuario único en el sistema
correo	VARCHAR(255)	NO	UNIQUE, NOT NULL, CHECK formato	Correo electrónico verificado
password_hash	VARCHAR(255)	NO	NOT NULL	Hash bcrypt de la contraseña (factor 12)
activo	BOOLEAN	NO	DEFAULT TRUE	Estado de la cuenta (activo/inactivo)
creado_en	TIMESTAMP WITH TIME ZONE	NO	DEFAULT NOW()	Fecha y hora de creación UTC
ultimo_login	TIMESTAMP WITH TIME ZONE	SÍ		Última fecha de autenticación exitosa
creado_por	UUID	SÍ	FK usuarios(id)	Usuario administrador que creó el registro

Tabla: reglas

Campo	Tipo	Nu lo	Restricción	Descripción
id	UUID	NO	PK	Identificador único de la regla
nombre	VARCHAR (200)	NO	NOT NULL	Nombre descriptivo de la regla
categoria	VARCHAR (100)	NO	NOT NULL	Categoría de negocio (inventario, demanda, riesgo...)
condicion	JSONB	NO	NOT NULL	Condición estructurada en formato JSON para el motor
accion	TEXT	NO	NOT NULL	Descripción de la acción/recomendación a ejecutar
peso	FLOAT	NO	DEFAULT 1.0, CHECK (0-10)	Peso de prioridad de la regla (0 = mínimo, 10 = máximo)
nivel_confianza	FLOAT	NO	DEFAULT 0.8, CHECK (0-1)	Nivel de confianza calculado por retroalimentación
version	VARCHAR (20)	NO	DEFAULT '1.0.0'	Versión semántica de la regla
estado	VARCHAR (20)	NO	CHECK (estado IN (...))	Estado: BORRADOR/PENDIENTE/ACTIVA/RECHAZADA/SUS PENDIDA/ARCHIVADA
conocimiento_id	UUID	SÍ	FK conocimientos(id)	Referencia al conocimiento fuente
creado_por	UUID	NO	FK usuarios(id)	Usuario que creó la regla
creado_en	TIMESTAMPZ	NO	DEFAULT NOW()	Fecha de creación
activada_en	TIMESTAMPZ	SÍ		Fecha de activación en el motor

13. Diseño del Backend

13.1 Estructura de Módulos

```

nexus-kbdss-backend/
├── main.py                                # Punto de entrada FastAPI
├── config/
│   ├── settings.py                       # Pydantic Settings (env vars)
│   └── database.py                       # SQLAlchemy engine & session
├── domain/
│   ├── entities/                        # Entidades de dominio
│   │   ├── usuario.py
│   │   ├── regla.py
│   │   ├── conocimiento.py
│   │   ├── escenario.py
│   │   ├── decision.py
│   │   └── kpi.py
│   ├── repositories/                   # Interfaces de repositorio
│   │   ├── i_usuario_repo.py
│   │   ├── i_regla_repo.py
│   │   └── ...otros
│   └── value_objects/
├── application/
│   ├── use_cases/                      # Casos de uso
│   │   ├── usuarios/
│   │   ├── conocimiento/
│   │   ├── reglas/
│   │   ├── analisis/
│   │   └── decisiones/
│   ├── dtos/                          # Data Transfer Objects
│   └── services/                      # Servicios de aplicación
├── infrastructure/
│   ├── database/
│   │   ├── models/                   # Modelos SQLAlchemy
│   │   └── repositories/             # Implementaciones de repositorio
│   ├── rule_engine/
│   │   ├── engine.py                 # Motor de reglas
│   │   └── condition_parser.py       # Parser de condiciones JSON
│   ├── migrations/                   # Alembic migrations
│   ├── cache/                        # Redis client
│   └── external/                     # Clientes servicios externos
└── presentation/
    ├── api/
    │   ├── v1/
    │   │   ├── routers/              # FastAPI routers
    │   │   │   ├── auth.py
    │   │   │   ├── users.py
    │   │   │   ├── knowledge.py
    │   │   │   ├── rules.py
    │   │   │   ├── escenarios.py
    │   │   │   ├── analysis.py
    │   │   │   ├── decisions.py
    │   │   │   ├── kpis.py
    │   │   │   ├── feedback.py
    │   │   │   └── reports.py
    │   │   └── schemas/              # Pydantic schemas

```

```
├── middleware/
│   ├── auth.py           # JWT middleware
│   ├── audit.py          # Auditoría automática
│   └── rate_limit.py     # Rate limiting
└── dependencies.py       # FastAPI dependencies (DI)
```

13.2 Flujo de Procesamiento de Decisiones

El flujo de procesamiento de una decisión asistida sigue estos pasos: (1) El Analista envía un escenario al endpoint de análisis; (2) el AnalysisService recupera todas las reglas activas de la base de datos; (3) el RuleEngine evalúa cada regla contra las variables del escenario; (4) las reglas que satisfacen sus condiciones generan recomendaciones; (5) el RecomendacionService prioriza y filtra las recomendaciones; (6) el resultado es cacheado en Redis (TTL 5 minutos) y retornado al cliente.

14. Arquitectura del Frontend

14.1 Estructura de Componentes

```
nexus-kbdss-frontend/  
├── src/  
│   ├── app/  
│   │   ├── router.tsx          # React Router v6 config  
│   │   ├── store.ts           # Zustand global store  
│   │   └── queryClient.ts      # TanStack Query client  
│   ├── features/              # Feature-based structure  
│   │   ├── auth/  
│   │   │   ├── LoginPage.tsx  
│   │   │   ├── AuthContext.tsx  
│   │   │   └── useAuth.ts  
│   │   ├── dashboard/  
│   │   │   ├── DashboardPage.tsx  
│   │   │   ├── widgets/  
│   │   │   │   ├── KPIWidget.tsx  
│   │   │   │   ├── TrendChart.tsx  
│   │   │   │   └── AlertsWidget.tsx  
│   │   ├── knowledge/  
│   │   │   ├── KnowledgeListPage.tsx  
│   │   │   ├── KnowledgeCaptureForm.tsx  
│   │   │   └── RuleConditionEditor.tsx  
│   │   ├── scenarios/  
│   │   │   ├── ScenarioListPage.tsx  
│   │   │   ├── ScenarioForm.tsx  
│   │   │   └── WhatIfAnalysisPage.tsx  
│   │   ├── decisions/  
│   │   │   ├── DecisionHistoryPage.tsx  
│   │   │   └── FeedbackForm.tsx  
│   │   └── admin/  
│   │       ├── UserManagementPage.tsx  
│   │       └── SystemConfigPage.tsx  
│   ├── shared/  
│   │   ├── components/        # Componentes reutilizables  
│   │   │   ├── Button.tsx  
│   │   │   ├── DataTable.tsx  
│   │   │   ├── Modal.tsx  
│   │   │   └── SemaforoIndicator.tsx  
│   │   ├── hooks/             # Custom hooks  
│   │   ├── utils/             # Utilidades  
│   │   └── types/              # TypeScript types  
│   └── api/                   # Axios API clients  
│       ├── axiosInstance.ts  
│       ├── authApi.ts  
│       ├── knowledgeApi.ts  
│       └── ...otros
```

14.2 Navegación y Rutas

Ruta	Componente	RoI Requerido
------	------------	---------------

/login	LoginPage	Público
/dashboard	DashboardPage	Todos los roles autenticados
/knowledge	KnowledgeListPage	Administrador de Conocimiento+
/knowledge/new	KnowledgeCaptureForm	Administrador de Conocimiento
/knowledge/:id	KnowledgeDetailPage	Administrador de Conocimiento+
/scenarios	ScenarioListPage	Analista+
/scenarios/:id/whatif	WhatIfAnalysisPage	Analista+
/decisions	DecisionHistoryPage	Decisor+
/decisions/:id/feedback	FeedbackForm	Decisor+
/kpis	KPIManagementPage	Analista+
/reports	ReportsPage	Analista+
/admin/users	UserManagementPage	Administrador del Sistema
/admin/roles	RoleManagementPage	Administrador del Sistema
/admin/system	SystemConfigPage	Administrador del Sistema
/audit	AuditLogPage	Administrador del Sistema

15. Especificación de APIs REST

15.1 Convenciones Generales

- Base URL: `https://api.nexuscorp.local/api/v1`
- Formato: JSON (Content-Type: `application/json`)
- Autenticación: Bearer Token (JWT) en header `Authorization`
- Versionado: URI versioning (`/api/v1/`, `/api/v2/`)
- Paginación: query params `page` (1-based) y `size` (default 20, max 100)
- Timestamps: ISO 8601 UTC (2025-01-15T10:30:00Z)
- IDs: UUID v4 en formato estándar

15.2 Endpoints Principales

Método	Endpoint	Auth Requerida	Descripción
POST	<code>/auth/login</code>	NO	Autenticación; retorna <code>access_token</code> y <code>refresh_token</code>
POST	<code>/auth/refresh</code>	Refresh Token	Renovar <code>access_token</code> usando <code>refresh_token</code>
POST	<code>/auth/logout</code>	JWT	Invalidar sesión y tokens
GET	<code>/users/</code>	ADMIN_SYS	Listar usuarios (paginado)
POST	<code>/users/</code>	ADMIN_SYS	Crear nuevo usuario
GET	<code>/users/{id}</code>	ADMIN_SYS	Obtener usuario por ID
PATCH	<code>/users/{id}</code>	ADMIN_SYS	Actualizar datos de usuario
DELETE	<code>/users/{id}</code>	ADMIN_SYS	Desactivar usuario (lógico)
GET	<code>/knowledge/</code>	ADMIN_KNOW	Listar conocimientos (filtros: estado, categoría)
POST	<code>/knowledge/</code>	ADMIN_KNOW	Capturar nuevo conocimiento
GET	<code>/knowledge/{id}</code>	ADMIN_KNOW	Detalle de conocimiento
PATCH	<code>/knowledge/{id}/validate</code>	ADMIN_KNOW	Aprobar/rechazar conocimiento
GET	<code>/rules/</code>	ADMIN_KNOW	Listar reglas (filtros: estado, categoría)

POST	/rules/	ADMIN_KNOW	Crear nueva regla
GET	/rules/{id}	ADMIN_KNOW	Detalle de regla con historial de versiones
PATCH	/rules/{id}/activate	ADMIN_KNOW	Activar regla en motor
POST	/rules/{id}/clone	ADMIN_KNOW	Clonar regla para nueva versión
GET	/scenarios/	ANALYST	Listar escenarios del usuario
POST	/scenarios/	ANALYST	Crear escenario nuevo
POST	/analysis/whatif	ANALYST	Ejecutar análisis What-If
GET	/analysis/{id}/results	ANALYST	Obtener resultados de análisis
GET	/recommendations/	DECISOR	Listar recomendaciones activas
PATCH	/recommendations/{id}/approve	DECISOR	Aprobar recomendación como decisión
GET	/decisions/	DECISOR	Historial de decisiones
POST	/feedback/	DECISOR	Registrar retroalimentación
GET	/kpis/	ANALYST	Listar KPIs con estado semáforo
POST	/kpis/	ADMIN_KNOW	Definir nuevo KPI
POST	/kpis/{id}/calculate	SYSTEM	Recalcular valor del KPI
GET	/reports/decisions	ANALYST	Generar reporte de decisiones (PDF/Excel)
GET	/audit/	ADMIN_SYS	Consultar log de auditoría
GET	/dashboard/summary	ALL_AUTH	Resumen ejecutivo para dashboard

15.3 Estructura de Respuestas Estándar

Respuesta exitosa (200/201):

```
{
  "success": true,
  "data": { ... },
  "message": "Operación completada exitosamente",
  "timestamp": "2025-01-15T10:30:00Z",
  "pagination": {
```

```
    "total": 150,  
    "page": 1,  
    "size": 20,  
    "pages": 8  
  }  
}
```

Respuesta de error (4xx/5xx):

```
{  
  "success": false,  
  "error": {  
    "code": "VALIDATION_ERROR",  
    "message": "Los datos de entrada no son válidos",  
    "details": [  
      { "field": "condicion", "message": "JSON inválido" }  
    ]  
  },  
  "timestamp": "2025-01-15T10:30:00Z",  
  "request_id": "req_abcl23xyz"  
}
```

15.4 Códigos de Error del Sistema

Código Error	HTTP Status	Descripción
AUTH_001	401	Token de autenticación ausente o inválido
AUTH_002	401	Token expirado; se requiere refresh
AUTH_003	403	Permisos insuficientes para la operación solicitada
VAL_001	422	Error de validación en datos de entrada
VAL_002	422	Condición JSON con sintaxis inválida para el motor de reglas
RES_001	404	Recurso no encontrado
RES_002	409	Conflicto: recurso ya existe (unicidad violada)
RES_003	409	Conflicto de regla: condición duplicada con regla activa existente
BUS_001	400	Error de regla de negocio: nivel de confianza insuficiente
BUS_002	400	Error de negocio: decisión requiere múltiples aprobadores
ENG_001	500	Error interno del motor de reglas
DB_001	503	Base de datos no disponible

RATE_001	429	Límite de peticiones excedido (rate limit)
----------	-----	--

16. Estrategia de Seguridad

16.1 Autenticación – OAuth2 con JWT

El KBDSS implementa el flujo OAuth2 Password Flow con tokens JWT (HS256). Al autenticarse exitosamente, el sistema emite dos tokens: un `access_token` (TTL: 8 horas) y un `refresh_token` (TTL: 7 días). El `refresh_token` se almacena en una cookie `HttpOnly`, `SameSite=Strict`, `Secure`; el `access_token` se retorna en el body de la respuesta y es almacenado en memoria por el frontend (no en `localStorage`).

```
# presentation/middleware/auth.py
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from config.settings import settings

oauth2_scheme = OAuth2PasswordBearer(tokenUrl='/api/v1/auth/login')

async def get_current_user(token: str = Depends(oauth2_scheme)) -> dict:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail={'code': 'AUTH_001', 'message': 'Token inválido'}
    )
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=['HS256'])
        user_id: str = payload.get('sub')
        if user_id is None: raise credentials_exception
        return {'user_id': user_id, 'roles': payload.get('roles', [])}
    except JWTError:
        raise credentials_exception
```

16.2 Autorización – Control de Acceso Basado en Roles (RBAC)

El sistema implementa RBAC con cuatro roles predefinidos. Los permisos son granulares a nivel de recurso y operación. La verificación de permisos se realiza mediante un decorador FastAPI en cada endpoint.

Módulo / Operación	Admin Sistema	Admin Conocimiento	Decisor	Analista
Gestión de Usuarios (CRUD)	✓	X	X	X
Gestión de Roles	✓	X	X	X
Captura de Conocimiento	✓	✓	X	✓
Validación de Reglas	✓	✓	X	X
Ejecutar Análisis What-If	✓	✓	✓	✓

Aprobar Recomendaciones	✓	X	✓	X
Registrar Retroalimentación	✓	X	✓	✓
Gestión de KPIs (CRUD)	✓	✓	X	✓ (Read)
Dashboard Ejecutivo	✓	✓	✓	✓
Log de Auditoría	✓	X	X	X
Configuración del Sistema	✓	X	X	X
Generación de Reportes	✓	✓	✓	✓

16.3 Protección de Datos y Auditoría

- Cifrado en tránsito: TLS 1.3 obligatorio; certificados Let's Encrypt gestionados por Certbot.
- Cifrado en reposo: contraseñas con bcrypt (factor 12); datos sensibles con pgcrypto AES-256.
- Auditoría automática: middleware de FastAPI intercepta todas las operaciones y genera entrada en tabla auditorias de forma atómica.
- Log de acceso: Nginx registra todas las peticiones con IP, timestamp, método, URI, status code y tiempo de respuesta.
- Política de sesión: máximo 3 sesiones concurrentes por usuario; sesiones antiguas invalidadas automáticamente.
- Política de contraseñas: mínimo 10 caracteres, al menos una mayúscula, un número y un carácter especial; rotación obligatoria cada 90 días.
- CORS: habilitado únicamente para el dominio oficial del frontend; no se permiten orígenes wildcard en producción.

17. Estrategia de Pruebas

17.1 Niveles de Prueba

Nivel	Herramienta	Alcance	Cobertura Objetivo
Pruebas Unitarias	pytest, unittest.mock	Entidades de dominio, casos de uso, motor de reglas	≥ 80%
Pruebas de Integración	pytest + TestClient FastAPI	Endpoints API + BD (PostgreSQL de prueba)	≥ 70%
Pruebas E2E	Playwright (TypeScript)	Flujos críticos de usuario en navegador real	Flujos críticos 100%
Pruebas de Rendimiento	k6, Locust	APIs bajo carga: 100 usuarios concurrentes	P95 ≤ 500ms
Pruebas de Seguridad	OWASP ZAP, Bandit (Python)	OWASP Top 10, análisis estático de código	Sin vulnerabilidades críticas/altas
Pruebas de Aceptación	Gherkin/Behave + usuarios reales	Criterios de aceptación de requerimientos funcionales	100% RF críticos

17.2 Casos de Prueba Funcionales Seleccionados

ID	RF	Escenario de Prueba	Entrada	Resultado Esperado
CP-01	RF-01	Crear usuario con datos válidos	POST /users/ con JSON válido	201 Created + DTO del usuario
CP-02	RF-01	Crear usuario con correo duplicado	POST /users/ con correo existente	409 Conflict, código RES_002
CP-03	RF-03	Crear regla con condición JSON válida	POST /rules/ con condición válida	201 Created, estado BORRADOR
CP-04	RF-03	Crear regla con condición inválida	POST /rules/ con JSON malformado	422, código VAL_002
CP-05	RF-05	Validar y activar regla pendiente	PATCH /rules/{id}/activate	200 OK, estado ACTIVA

CP-06	RF-07	Análisis What-If con 1 regla activa	POST /analysis/whatif con escenario	200 OK con recomendación generada
CP-07	RF-07	Análisis sin reglas activas	POST /analysis/whatif sin reglas	200 OK con lista vacía + alerta
CP-08	RF-09	KPI supera umbral crítico	KPI calculado > umbral_critical	Alerta generada, estado semáforo ROJO
CP-09	RF-10	Retroalimentación negativa actualiza confianza	POST /feedback/ resultado negativo	nivel_confianza reducido en regla
CP-10	RF-12	Toda operación CRUD genera auditoría	Cualquier POST/PATCH/DELETE	Entrada en tabla auditorias
CP-11	RF-11	Acceso a historial por usuario no autorizado	GET /decisions/ sin JWT	401 Unauthorized
CP-12	RF-14	Dashboard carga todos los widgets	GET /dashboard/summary	200 OK con todos los KPIs y alertas

17.3 Matriz de Trazabilidad de Requisitos

RF	Nombre	Caso de Uso	Caso de Prueba	Módulo Backend	Tabla BD
RF-01	Gestión Usuarios	UC-01	CP-01, CP-02	UserService	usuarios, roles
RF-02	Gestión Roles	UC-02	–	RoleService	roles, permisos
RF-03	Gestión Reglas	UC-03	CP-03, CP-04	RuleService	reglas
RF-04	Captura Conocimiento	UC-04	–	KnowledgeService	conocimientos
RF-05	Validación Conocimiento	UC-05	CP-05	KnowledgeService	reglas
RF-06	Gestión Escenarios	UC-06	–	ScenarioService	escenarios

RF-07	Análisis What-If	UC-07	CP-06, CP-07	AnalysisService	escenarios, recomendaciones
RF-08	Generación Recomend.	UC-07	–	RuleEngine	recomendaciones
RF-09	Gestión KPIs	UC-09	CP-08	KPIService	kpis
RF-10	Retroalimentación	UC-10	CP-09	FeedbackService	retroalimentaciones
RF-11	Historial Decisiones	UC-11	CP-11	DecisionService	decisiones
RF-12	Auditoría	UC-12	CP-10	AuditService	auditorias
RF-13	Reportes	–	–	ReportService	múltiples
RF-14	Dashboard Ejecutivo	–	CP-12	DashboardService	múltiples
RF-15	Administración	UC-15	–	AdminService	múltiples

18. Glosario Técnico

Término	Definición
Aprendizaje Organizacional	Proceso mediante el cual una organización mejora su capacidad de acción a través de la adquisición, interpretación y aplicación de conocimiento.
Base de Conocimiento	Repositorio estructurado de reglas, hechos y relaciones que representan el conocimiento organizacional formalizado.
Cadena de Suministro	Red de organizaciones, recursos y procesos involucrados en la creación y distribución de un producto hasta el consumidor final.
Ciclo de Vida del Conocimiento	Secuencia de etapas que atraviesa el conocimiento organizacional: captura, validación, activación, aplicación, retroalimentación y archivo.
Decisión Asistida	Decisión tomada por un ser humano con apoyo de recomendaciones generadas por el sistema, sin que el sistema tome la decisión autónomamente.
Efectividad de Regla	Métrica que mide el porcentaje de recomendaciones generadas por una regla que produjeron el resultado esperado, calculada a partir de la retroalimentación histórica.
Encadenamiento Hacia Adelante	Estrategia de inferencia del motor de reglas que parte de los hechos conocidos y aplica reglas para derivar nuevas conclusiones.
Escenario Hipotético	Conjunto de valores asumidos para variables de negocio con el propósito de simular condiciones futuras o alternativas y evaluar su impacto.
Hecho (Fact)	Valor conocido y verificable de una variable de negocio en un momento dado, utilizado como entrada para la evaluación de reglas.
Inferencia	Proceso mediante el cual el motor de reglas deriva conclusiones (recomendaciones) a partir de hechos y reglas de conocimiento.
Inventario de Baja Rotación	SKU cuya tasa de venta en un período definido es significativamente menor al promedio histórico, indicando posible obsolescencia o sobrestock.
KPI Semáforo	Sistema visual de tres colores (verde/amarillo/rojo) que indica si un indicador está dentro de su rango óptimo, en zona de alerta o en estado crítico.
Motor de Inferencia	Componente de software que aplica las reglas de la base de conocimiento sobre los hechos disponibles para generar conclusiones o recomendaciones.
Nivel de Confianza	Valor entre 0 y 1 que representa la probabilidad de que una regla genere recomendaciones efectivas, calculado a partir de la retroalimentación histórica.

Regla de Producción	Estructura lógica de la forma IF condición THEN acción utilizada en los sistemas de producción y motores de reglas.
SKU	Stock Keeping Unit. Unidad de mantenimiento de stock; identificador único asignado a un producto para su gestión de inventario.
SLA	Service Level Agreement. Acuerdo de nivel de servicio que define métricas mínimas de rendimiento entre proveedor y cliente.
Versionado Semántico	Estándar de nomenclatura de versiones (MAJOR.MINOR.PATCH) que comunica el tipo de cambios entre versiones de un componente de software.
What-If Analysis	Técnica analítica que evalúa el impacto de cambios hipotéticos en variables de entrada sobre los resultados del sistema o negocio.

19. Conclusiones Técnicas

La especificación técnica de la Fase 2 del KBDSS de Nexus-Corp establece una base sólida y exhaustiva para el desarrollo de un sistema de gestión del conocimiento organizacional de nivel empresarial. Las decisiones arquitectónicas tomadas – Clean Architecture, separación en capas bien definidas, API REST versionada y base de datos relacional robusta – garantizan que el sistema sea mantenible, escalable y evolucionable a largo plazo.

El motor de reglas de encadenamiento hacia adelante, aunque implementado inicialmente de forma simplificada, provee la base computacional necesaria para la evaluación determinística de reglas de negocio, con mecanismos de versionado y control de confianza que permiten el aprendizaje organizacional continuo sin pérdida de trazabilidad.

La estrategia de seguridad multi-capas (TLS, JWT, RBAC, auditoría, rate limiting) cumple con los estándares OWASP y las políticas internas de Nexus-Corp, garantizando la confidencialidad, integridad y disponibilidad de la información de conocimiento organizacional.

El modelado UML completo (casos de uso, clases, secuencia, actividades, componentes y despliegue) proporciona artefactos de comunicación técnica precisos que facilitan la coordinación entre equipos de desarrollo, QA, infraestructura y negocio durante todas las fases de construcción del sistema.

20. Recomendaciones Técnicas

- Fase de Desarrollo: Implementar el backend utilizando TDD (Test-Driven Development) con pytest, garantizando que cada caso de uso tenga pruebas unitarias antes de la implementación. Esto reducirá el costo de corrección de errores y asegurará la cobertura mínima del 80%.
- Motor de Reglas: Evaluar la migración a un motor de reglas de producción (Drools, OpenL Tablets) en la Fase 3 si el volumen de reglas supera las 5,000 reglas activas o si se requieren capacidades de encadenamiento hacia atrás.
- Caché: Implementar caché Redis para los resultados del análisis What-If y los valores de KPIs desde la primera versión. El impacto en rendimiento es significativo cuando múltiples usuarios ejecutan análisis simultáneos.
- Migraciones: Establecer un protocolo estricto de revisión de migraciones Alembic antes de cada despliegue. Las migraciones deben ser reversibles y probadas en un ambiente de staging antes de producción.
- Monitoreo: Configurar alertas de Grafana para los indicadores críticos desde el primer despliegue: tiempo de respuesta P95 > 500ms, tasa de errores 5xx > 1%, uso de CPU > 80% y espacio en disco > 85%.
- Escalabilidad: Diseñar el AnalysisService para ser stateless desde el inicio, permitiendo ejecutar múltiples instancias del backend detrás de Nginx con balanceo de carga round-robin sin modificaciones.
- Documentación: Generar y publicar automáticamente la documentación OpenAPI 3.1 con cada despliegue. Configurar Redoc como interfaz de documentación pública para integradores internos.
- Capacitación: Planificar sesiones de capacitación de 4 horas para Administradores de Conocimiento antes del arranque productivo, con énfasis en el proceso de captura y validación de reglas y en la interpretación de niveles de confianza.

- Revisión Continua: Establecer un comité mensual de revisión de la base de conocimiento para identificar reglas con baja confianza, reglas obsoletas y oportunidades de captura de nuevo conocimiento organizacional.

—— *Fin del Documento Técnico – Fase 2* ——

Nexus-Corp – Arquitectura de Inteligencia Organizacional | ETS-KBDSS-F2-2025 | v2.0.0